A Distributed Real-Time Intrusion Detection System on a Jetson Orin Nano Super Edge Cluster using Kafka and PySpark

Thai Nguyen Vu[1] Dung Van Bui[2] Nhut Tri Do[2] Van Du Nguyen[3]

SOICT 2026

[1]Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam
[2]University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam
[3]Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

I. PROBLEM AND CONTRIBUTIONS

Edge intrusion detection: three open gaps

Detectors now *fit* on single-board hardware — but a monolithic one classifies **every** flow and saturates an 8 GB ARM board.

What the literature does not tell us

1. Everything is measured on a **single device**.
2. Throughput, latency and **energy** are never reported together.
3. Offline scores come from **leaky protocols**.

Our question

How should inference be *placed* across a constrained two-node edge cluster?

Our answer

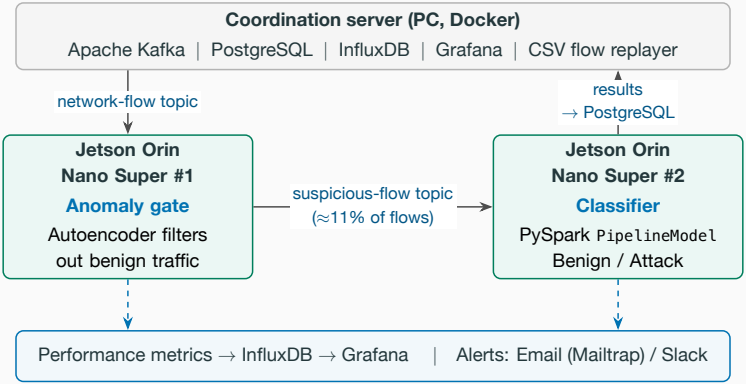
Split it: a cheap **autoencoder gate** on board #1, the heavy **PySpark classifier** on board #2, joined by a Kafka topic.

Contributions

1. A **two-stage distributed IDS**: anomaly gate + Spark classifier, physically separated across two boards.
2. **Three deployment modes** for a two-node Jetson cluster — pipeline split, horizontal Kafka scaling, Spark standalone cluster.
3. An **end-to-end evaluation**: throughput, latency percentiles, per-node CPU/RAM *and* energy per verdict.
4. **Leakage-aware provenance** — the deployed model is exactly the one our companion study selected under a leakage-free protocol.

II. SYSTEM AND EVALUATION SETUP

Split-deployment architecture



The gate scores all 77 features; its threshold is fixed *offline* on benign validation data, never on the test set. Batches of 20, or after 1 s.

CICIDS2017: Sharafaldin et al., ICISSP 2018 · Kafka: Kreps et al., NetDB 2011 · Spark: Zaharia et al., HotCloud 2010.

Deployment modes and what we measure

Single node full pipeline on one board, gate off or on.

A: Pipeline split gate on #1, classifier on #2 (our design).

B: Horizontal both boards run the full role (gate off).

C: Spark cluster host as master, both boards as workers.

Every flow is followed from send to verdict, queueing included. **Sustained capacity**: the highest rate at which every flow gets its verdict within 10 s of the load.

Setup

2× Jetson Orin Nano Super
(6-core A78AE, 8 GB, ≈67 TOPS)

PySpark 3.4.1, Kafka 3.5, Wi-Fi LAN

10,000 test-split flows, 15% attacks

Rate sweep 1–40 flows/s, 2 runs each

Energy via `tegrastats`

III. RESULTS AND CONCLUSION

Result 1: splitting the pipeline sustains 9× the load

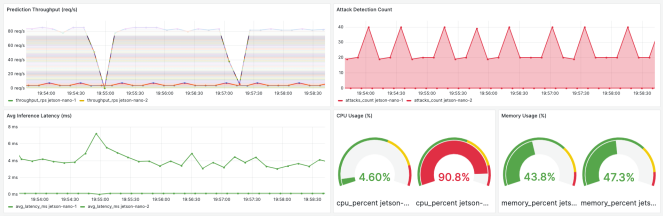
Mode	Flows/s	e2e p95 (s)	J / verdict
Single node	2.5	11.3	3.04
Single node + gate	≈2.5	7.8	3.04
B: Horizontal	3.3	8.2	3.20
C: Spark cluster	9–17	5.2	1.15
A: Split	**22.4**	**7.1**	**0.60**

Flows/s: sustained, every flow done within 10 s. **e2e p95**: send to verdict, queueing included.

- **Split: 9× a single node, 5× less energy per verdict.**
- **The same gate on the same board gains nothing**: the second board is what pays.
- Tail latency is seconds: one PySpark batch takes seconds.

Pipeline split is the recommended default: highest sustained load at the lowest energy per verdict.

Result 2: the load asymmetry is visible live



At peak load, split mode (earlier run, previous model):

- gate node: **4.6% CPU**
- classifier node: **90.8% CPU**

The classifier dominates the compute budget — which is exactly why moving it off the ingest path pays.

Result 3: what the gate lets through

Gate threshold (benign quantile)	Attacks forwarded	False alarms	Flows skipped
0.99	72.4%	0.99%	88.3%
0.995 (deployed)	**68.1%**	**0.49%**	89.4%
0.999	39.8%	0.09%	94.0%

Offline, CICIDS2017 test split; gate on all 77 features.
Streamed live: 69.6% of 1,261 attack flows caught.

- A flow the gate skips is final: **about 30% of attack flows** never reach the classifier.
- On the classifier's 30 features the gate would forward only 44.8%.
- The operator picks the point that fits the budget.

The capacity gain is paid for in recall: the gate is a **tunable trade-off**, not a free filter.

Result 4: the price of one unified stack

We serve with Spark for *consistency with distributed training*. What does that cost? The *deployed forest*, exported, same board:

Engine	Flows/s	Batch p95 (ms)	mJ per flow
PySpark (deployed)	1.4	7515	1506
NumPy	54.8	194	11.3
ONNX Runtime	**10,092**	**1.2**	**0.087**

Batch 10, as deployed; identical predictions on every flow. mJ per flow: energy above the board's idle power. A Spark call costs $\approx$7 s at any batch up to 1,000, so at batch 10,000 the gap is $\approx$27$\times$ (backup).

ONNX Runtime serves the same forest $\approx$**7,200**$\times$ faster. Target for the next iteration: **train on Spark, serve with ONNX/TensorRT**.

Conclusion

What we showed

- A reproducible distributed edge IDS: Kafka + PySpark on a Jetson Orin Nano Super cluster.
- Gate and classifier on separate boards: **2.5 → 22.4 flows/s** sustained, e2e p95 ≈7 s.
- Energy per verdict: **3.0 J → 0.60 J**.
- ONNX serves the same forest ≈7,200× faster.

Limitations

- Two-node lab cluster over Wi-Fi; replayed traffic.
- Gate skips ≈30% of attack flows.
- GPU unused (CPU-only pipeline); evasion untested.

Next

- ONNX/TensorRT serving; live traffic; retraining under drift.

Thank you — questions?

Key references

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Shi, W. et al. *Edge computing: Vision and challenges*. IEEE Internet of Things Journal 3(5), 2016. **edge setting**

[3] Chandola, V., Banerjee, A. & Kumar, V. *Anomaly detection: A survey*. ACM Computing Surveys 41(3), 2009. **anomaly gate**

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP Top-30**

[5] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark**

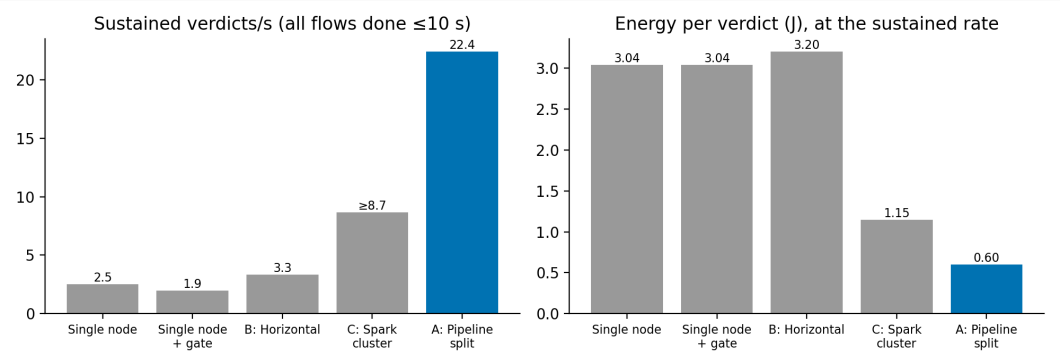
[6] Kreps, J., Narkhede, N. & Rao, J. *Kafka: a Distributed Messaging System for Log Processing*. NetDB 2011. **Kafka**

[7] Mohan Kiran Kumar, K. et al. *Distributed Intrusion Detection System using Kafka and Spark Streaming*. ICVADV 2025. **closest system**

[8] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **leakage-aware protocol**

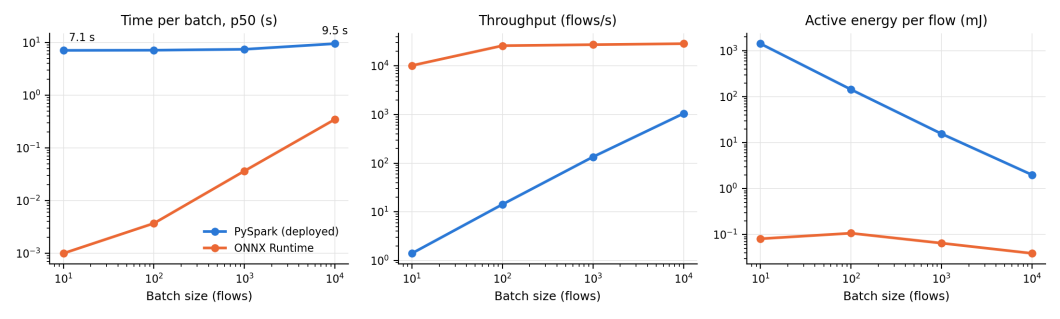
Backup slides

Backup: sustained capacity and energy per mode



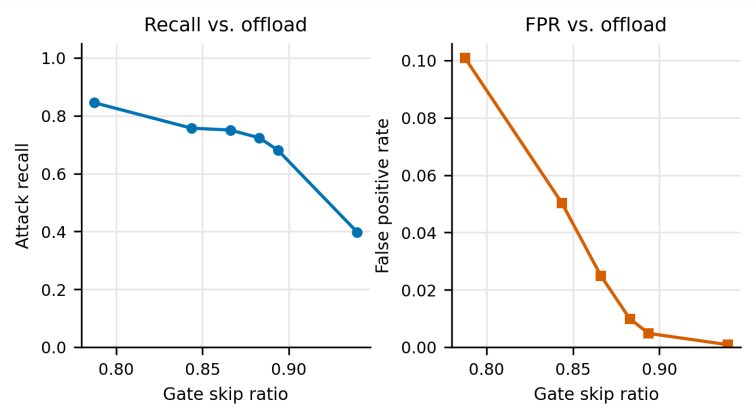
Sustained verdicts/s with every flow done within 10 s (left); energy per verdict at that rate (right).

Backup: Spark pays per call, not per tree



Same forest, same board, batch 10 to 10,000. A PySpark call takes 7.1–7.4 s up to batch 1,000 and 9.5 s at 10,000; ONNX saturates near 28,000 flows/s. The gap falls from ≈7,200× (batch 10) to ≈27× (batch 10,000).

Backup: the gate is an operating curve, not a point



Attack recall (left) and false-positive rate (right) vs. gate-skip ratio. Raising the skip ratio offloads more Spark work and costs recall; the operator chooses the point.

Backup: which model, and why

Configuration	F1	Features	Size (MB)	Edge rationale
Baseline + XGBoost	0.9971	77	2.32	Reference accuracy
SHAP Top-30 + XGBoost	0.9970	30	0.003†	Explainable; deployed feature set
RF Top-30 + XGBoost	0.9922	30	2.52	Embedded ranking
PCA k=40 + XGBoost	0.9939	40	3.82	Unsupervised reduction

We deploy **SHAP Top-30 with Random Forest** (exported model: test F1 0.9959, 30 features): a native Spark ML estimator, so the `PipelineModel` loads on ARM64 without the native libraries Spark's XGBoost integration needs. The SHAP ranking is computed after `destination_port` is removed.

†Distributed saving under-reports size; true size ≈2.5 MB.

Backup: full monitoring dashboard



Per-node throughput, detected attacks, inference latency, CPU/RAM for both boards, PostgreSQL-backed alerts.

Backup: measurement protocol

- Host-side orchestration starts pipeline roles on the Jetsons over SSH, drives the load, collects every verdict.
- Each verdict stores the flow's send time and replay row: latency is send to verdict, *queueing included*; skip ratio and attack recall come from the flows themselves.
- Rate sweep per mode; a rate is sustained when every flow sent in the 45 s window gets its verdict within 10 s after it. 2 runs per rate.
- Fresh Kafka consumer group per run, so no run inherits a backlog.
- Tiers batch 20 flows or flush after 1 s.
- Energy: `tegrastats`, 30 s idle baseline then load at the sustained rate; two-node modes sum both boards.